```
    setTicketsToBuyCount(parseInt(target.value, 10))
  }

  return (
    <div>
      <span>How many tickets would you like?</span>
      <span className="select">
        <select onChange={setTicketsOnChange}>
          {options(seatsPerRow)}
        </select>
      </span>
    </div>
  )
}

export default VenueHeader
```

The incoming props are declared first. The props are the number of seats in a row and the `setTicketsToBuy` function, getting passed down from the parent class so it can be invoked from the header.

Then, we build up the list of `<option>` tags the same way we needed to build up the list of rows. We do that with a `map` function that returns a JSX representation of the option, including a `key` because it's a React list.

The return value of the component is the JSX for the header—some text and a `<select>` tag containing our previously created options. The `select` item has an `onChange` event handler, which triggers a `setTicketsOnChange` function.

In that function, which is nested inside the component so it has access to the `setTicketsToBuy` prop, we extract the new value of the select tag and pass it to our `setTicketsToBuy` setter function, which, since it's a `useState` setter function, triggers a change in state, which triggers a draw of components that used that state.

Next, the `VenueBody` component calls the list of rows as before, except we are passing the number of tickets to buy as a new prop in the object. Because `ticketsToBuy` is passed as a prop, changing its value with the `setTicketsToBuy` setter will cause those components to re-render. Keep in mind that calling the component function again with new prop values does not re-initialize React's internal state